EX.NO:
DATE: **PRINCIPAL COMPONENT ANALYSIS**

AIM:

To apply Principal Component Analysis (PCA) on the Breast Cancer Wisconsin dataset in order to reduce the dimensionality of the dataset while retaining most of the important information (variance)

DATASET DESCRIPTION:

The Breast Cancer Wisconsin dataset is used for breast cancer diagnosis. It contains information computed from digitized images of fine needle aspirates (FNA) of breast masses.

The dataset consists of **569 samples** and **30 numerical features**. These features describe characteristics of the cell nuclei such as:

- Radius
- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity
- Symmetry
- Fractal dimension

Each sample is classified into one of two categories:

- **Malignant (M)** – Cancerous tumor
- **Benign (B)** – Non-cancerous tumor

This dataset is commonly used for classification and dimensionality reduction techniques like PCA because it contains many correlated features.

ALGORITHM:

Step 1: Import the required libraries.

Step 2: Load the Breast Cancer Wisconsin dataset.

Step 3: Remove unnecessary columns such as id and empty columns.

OUTPUT:

```
PS E:\PRINCIPALCOMPONENTANALYSIS> python main.py
Loading dataset...
Original Dataset Shape: (569, 33)
         id diagnosis  radius_mean  ...  symmetry_worst  fractal_dimension_worst  Unnamed: 32
0    842302         M        17.99  ...          0.4601                  0.11890          NaN
1    842517         M        20.57  ...          0.2750                  0.08902          NaN
2  84300903         M        19.69  ...          0.3613                  0.08758          NaN
3  84348301         M        11.42  ...          0.6638                  0.17300          NaN
4  84358402         M        20.29  ...          0.2364                  0.07678          NaN

[5 rows x 33 columns]

Features Shape: (569, 30)
Target Shape: (569,)

Data Standardized.

Explained Variance Ratio:
[4.42720256e-01 1.89711820e-01 9.39316326e-02 6.60213492e-02
 5.49576849e-02 4.02452204e-02 2.25073371e-02 1.58872380e-02
 1.38964937e-02 1.16897819e-02 9.79718988e-03 8.70537901e-03
 8.04524987e-03 5.23365745e-03 3.13783217e-03 2.66209337e-03
 1.97996793e-03 1.75395945e-03 1.64925306e-03 1.03864675e-03
 9.99096464e-04 9.14646751e-04 8.11361259e-04 6.01833567e-04
 5.16042379e-04 2.72587995e-04 2.30015463e-04 5.29779290e-05
 2.49601032e-05 4.43482743e-06]

Original Shape: (569, 30)
Reduced Shape after PCA: (569, 10)
```

```
==============================
Results WITHOUT PCA
==============================
Accuracy: 0.9649122807017544
              precision    recall  f1-score   support

           0       0.98      0.93      0.95        43
           1       0.96      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114


==============================
Results WITH PCA
==============================
Accuracy: 0.956140350877193
              precision    recall  f1-score   support

           0       0.93      0.95      0.94        43
           1       0.97      0.96      0.96        71

    accuracy                           0.96       114
   macro avg       0.95      0.96      0.95       114
weighted avg       0.96      0.96      0.96       114


Program Completed Successfully ✅
```

Step 4: Convert the target variable (M and B) into numeric form.

Step 5: Separate the feature matrix (X) and target variable (y).

Step 6: Standardize the feature values using StandardScaler so that each feature has mean 0 and standard deviation 1.

Step 7: Apply PCA on the standardized data.

Step 8: Compute the explained variance ratio of each principal component.

Step 9: Select the number of principal components that retain 95% of the total variance.

Step 10: Transform the original dataset into the reduced principal component space.

PROGRAM:

```
# ================================
# PCA Implementation on
# Breast Cancer Wisconsin Dataset
# ================================

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# ------------------------------
# 1. Load Dataset
# ------------------------------
print("Loading dataset...")

df = pd.read_csv("data.csv")

print("Original Dataset Shape:", df.shape)
print(df.head())

# ------------------------------
# 2. Data Preprocessing
# ------------------------------

# Drop unnecessary columns
df = df.drop(['id', 'Unnamed: 32'], axis=1)
```

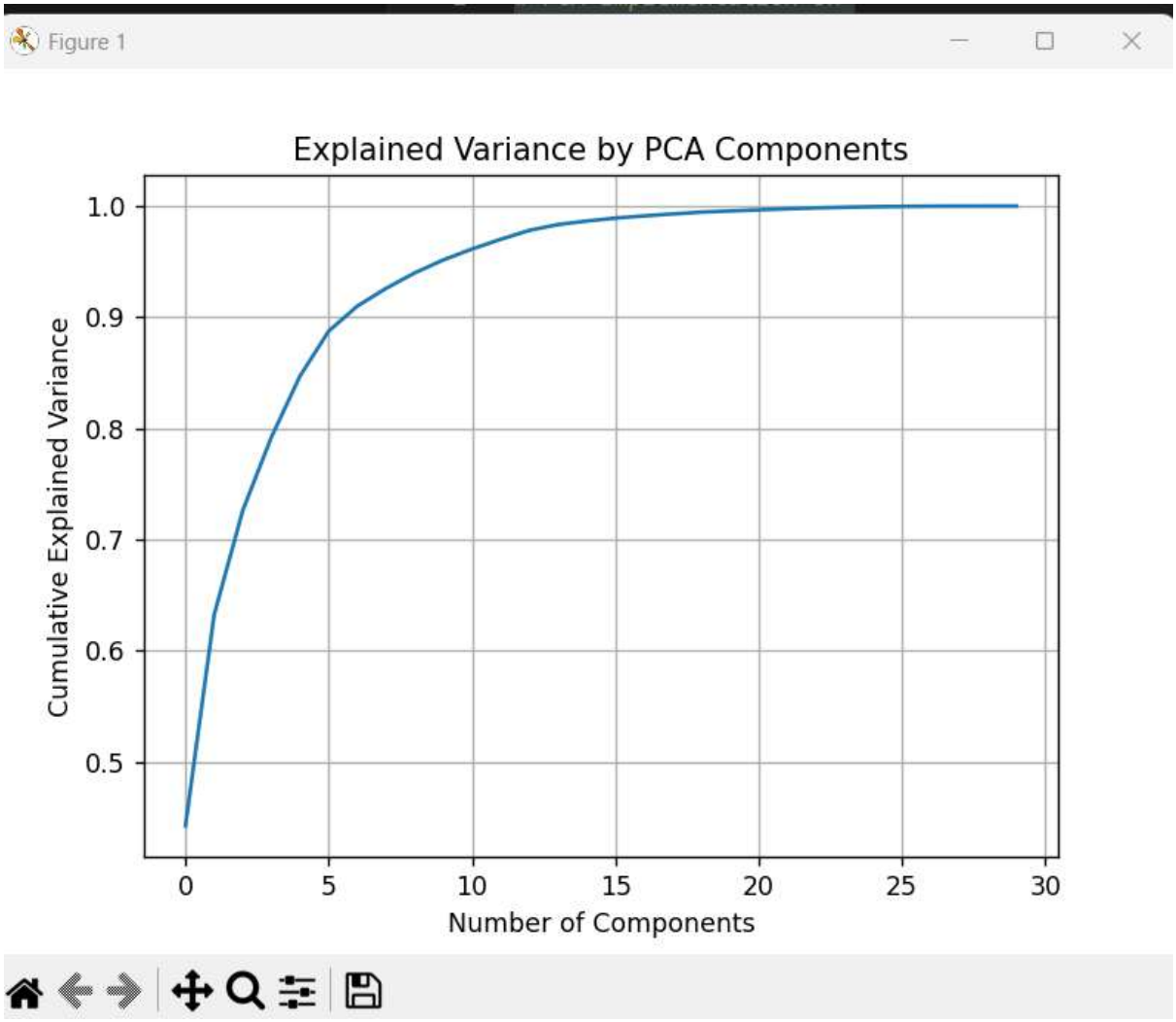
Figure 1
Explained Variance by PCA Components
Cumulative Explained Variance
1.0
0.9
0.8
0.7
0.6
0.5
0
5
10
15
20
25
30
Number of Components

```
# Convert target variable to numeric
df['diagnosis'] = df['diagnosis'].map({'M': 0, 'B': 1})

# Separate features and target
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

print("\nFeatures Shape:", X.shape)
print("Target Shape:", y.shape)

# ------------------------------
# 3. Standardization (Important for PCA)
# ------------------------------

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nData Standardized.")

# ------------------------------
# 4. Apply PCA (All Components)
# ------------------------------

pca_full = PCA()
X_pca_full = pca_full.fit_transform(X_scaled)

# Plot Explained Variance
plt.figure()
plt.plot(np.cumsum(pca_full.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by PCA Components')
plt.grid()
plt.show()

# Print variance values
print("\nExplained Variance Ratio:")
print(pca_full.explained_variance_ratio_)

# ------------------------------
# 5. Reduce to 95% Variance
# ------------------------------

pca = PCA(n_components=0.95)  # Automatically selects components
X_pca = pca.fit_transform(X_scaled)

print("\nOriginal Shape:", X_scaled.shape)
print("Reduced Shape after PCA:", X_pca.shape)
```

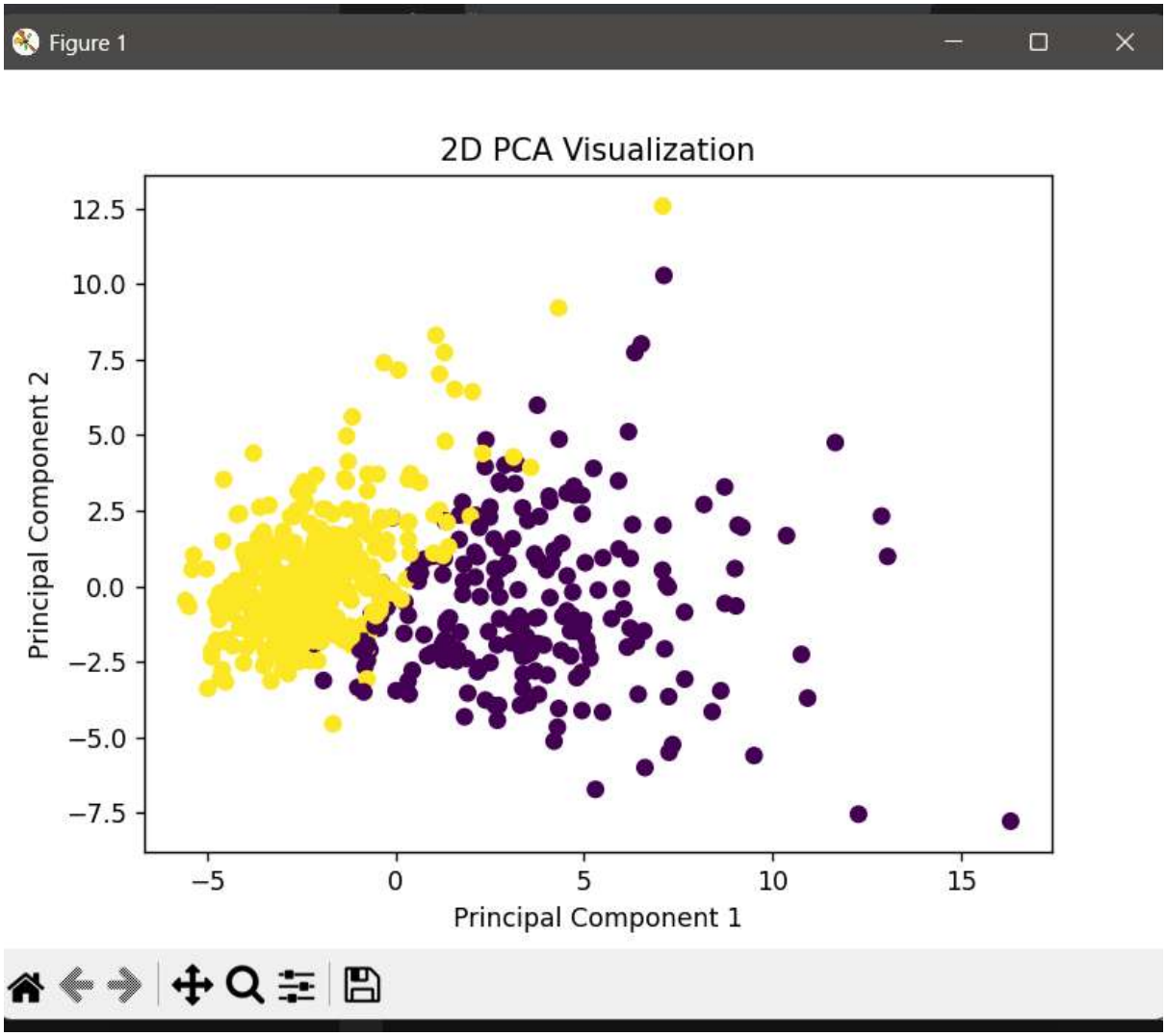
Figure 1
2D PCA Visualization
Principal Component 2
Principal Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15

```
# ------------------------------
# 6. 2D Visualization
# ------------------------------

pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)

plt.figure()
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("2D PCA Visualization")
plt.show()

# ------------------------------
# 7. Model WITHOUT PCA
# ------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n================================")
print("Results WITHOUT PCA")
print("================================")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# ------------------------------
# 8. Model WITH PCA
# ------------------------------

X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42)

model_pca = RandomForestClassifier(random_state=42)
model_pca.fit(X_train_pca, y_train)

y_pred_pca = model_pca.predict(X_test_pca)
print("\n================================")
print("Results WITH PCA")
print("================================")
print("Accuracy:", accuracy_score(y_test, y_pred_pca))
print(classification_report(y_test, y_pred_pca))
```

```
print("\nProgram Completed Successfully ✅ ")
```

RESULT:

After applying Principal Component Analysis (PCA) to the Breast Cancer Wisconsin dataset, the original 30 features were reduced to approximately 10 principal components while retaining 95% of the total variance.

The cumulative explained variance graph showed that most of the important information in the dataset is captured within the first few principal components.

Thus, PCA successfully reduced the dimensionality of the dataset without significant loss of information, making the data simpler and more efficient for further analysis.

GitHub repo link:

VarshiniRamesh2005/PCA-Project